

数据读取

数据读取

- 1. 快速上手
- 2. 硬件接线
- 3. 使用方法
- 4. 现象结果
- 5. 代码解释

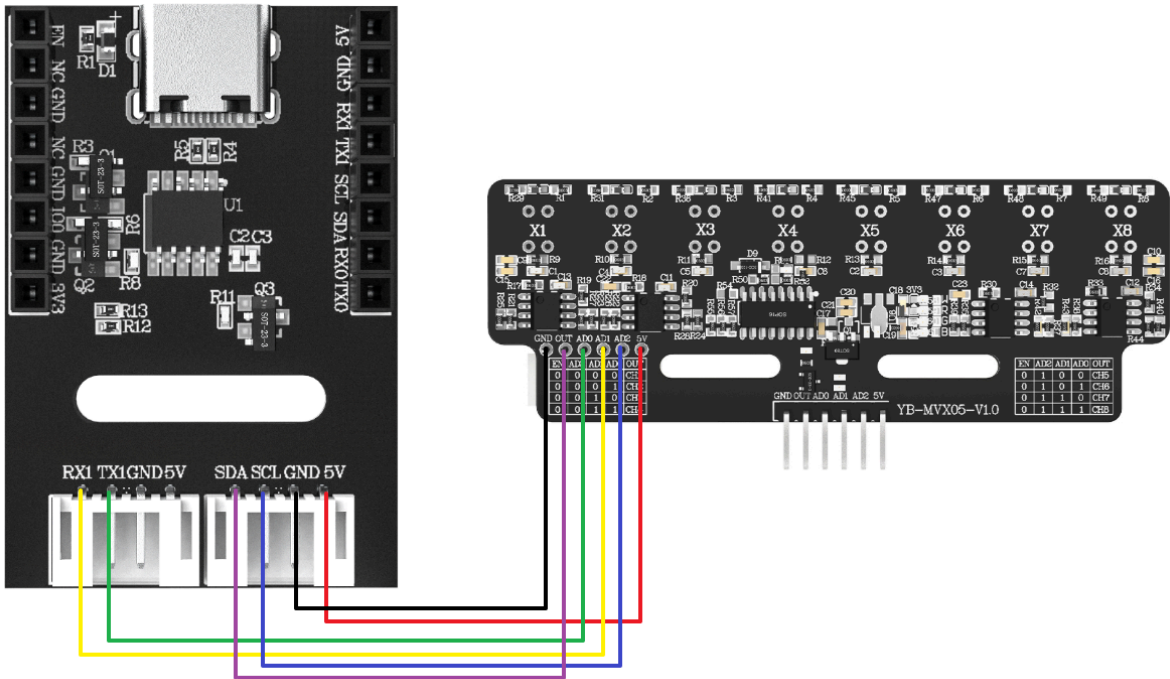
1. 快速上手

本教程讲解如何使用 ESP32-S3 主板读取八路灰度巡线模块的数字量信息，并通过串口助手打印结果。

教程中使用的硬件均是亚博售卖的 ESP32-S3 WIFI图传模块lite、八路灰度巡线模块。查看下方的接线图进行接线后，再给 ESP32-S3 烧录程序，就可以获得数据了。非亚博的模块仅供参考。

2. 硬件接线

ESP32-S3 的 USB 口需要使用数据线连接到电脑的 USB 端口。



亚博售卖的八路灰度模块连接使用的线材为XH2.54转6pin杜邦线：

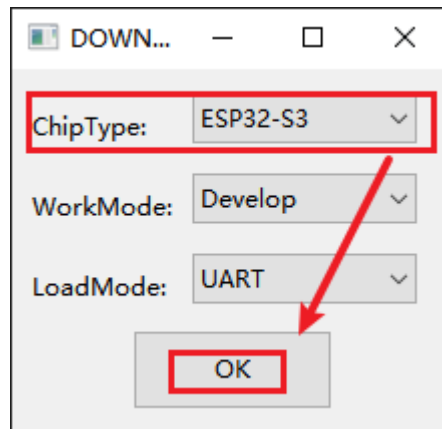


八路灰度巡线模块	ESP32-S3
5V	5V
GND	GND
AD0	RX1(35)
AD1	TX1(36)
AD2	SCL(37)
OUT	SDA(38)

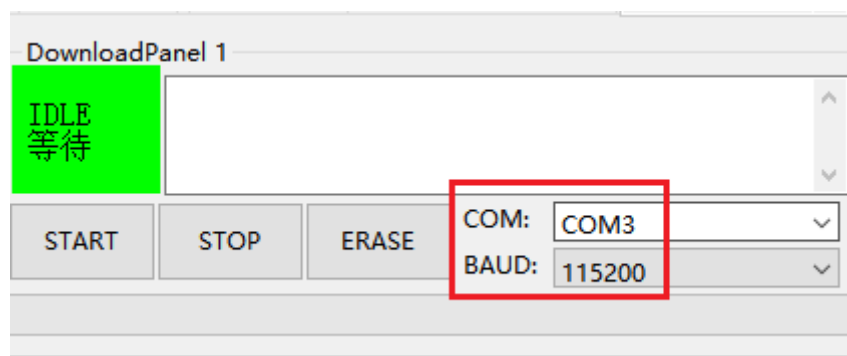
这里的串口和IIC标识只是使用的ESP32模块板子上的标识，实际上并未使用串口和IIC功能。

3. 使用方法

1. 打开烧录工具 `flash_download_tool`，芯片选择ESP32-S3，然后就点击OK。

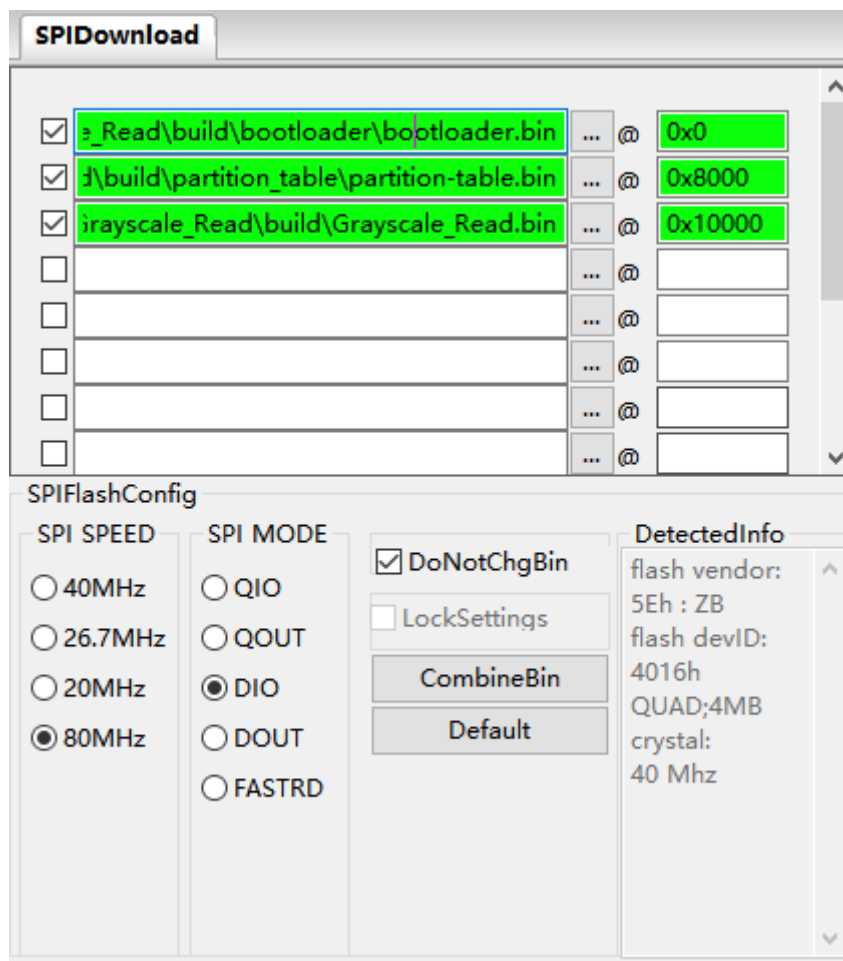


2. 通过数据线将 ESP32-S3 连接到电脑，然后软件里面选择好COM口和波特率。



3. 在 `SPIDownload` 窗口选择三个要烧录的ESP32S3的固件，路径一般在代码工程根目录下的 `build` 文件夹里，格式为bin文件，固件地址记得输入，必须确认勾选bin文件左边的方框，并且文件和地址都为绿色背景。

勾选DoNotChgBin，其他配置保持默认即可。

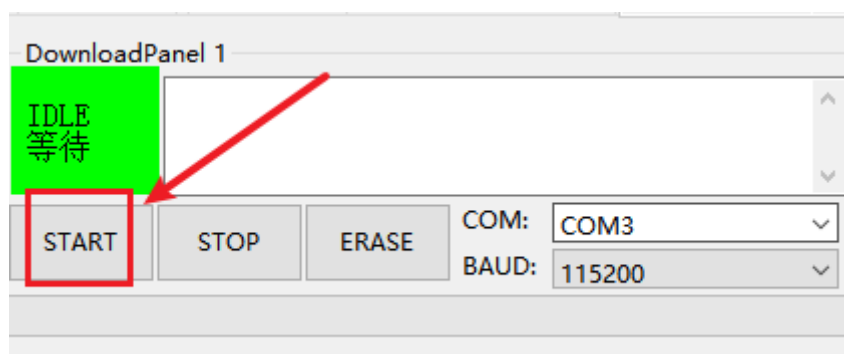


文件与地址对应关系如下表所示：

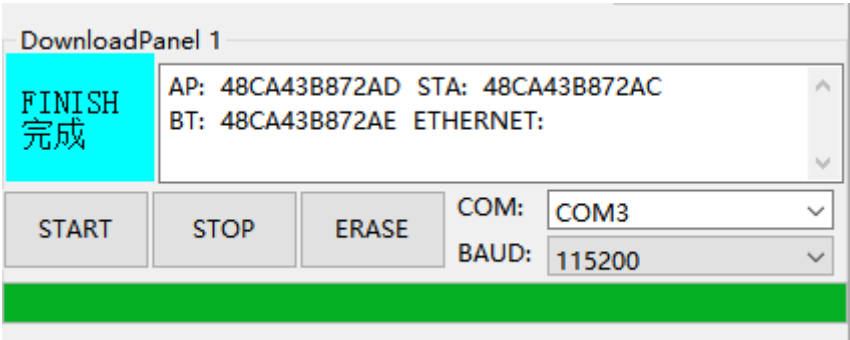
固件名称	固件地址	备注
bootloader.bin	0x0000	引导文件
partition-table.bin	0x8000	分区表文件
Grayscale_Read.bin	0x10000	功能文件

4. 点击START按钮，工具即自动开始烧录固件。

注：如果没有自动开始烧录固件，请先按住IO0键，再按EN键，然后同时松开，手动进入烧录模式，然后再点击START按钮。



5. 下载完成后，提示蓝色FINISH标识。此时断电重启单片机或者按一下EN键重启程序，关闭烧录工具。最后使用串口助手打开此端口就可以看到打印信息了。



4. 现象结果

烧录完程序后，在串口助手打开。您可以看到持续打印的八路灰度巡线模块的数字量。X1 对应模块上的 X1，当 X1 的灯亮起时，对应的值就为 1。

```
[2025-11-08 17:18:48.215]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.312]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.407]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.502]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.613]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.709]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]
```

5. 代码解释

```
// Grayscale_Read.c
void app_main(void)
{
    Grayscale_Sensor_Init();

    xTaskCreate(
        Grayscale_ReadData_Task,    // 任务函数
        "GrayscaleReadData",       // 任务名称
        4096,                      // 堆栈大小（字节）
        NULL,                      // 优先级
        2,                         // 优先级
        NULL
    );
}
```

```

void Grayscale_ReadData_Task(void *arg) {
    while (1) {
        Grayscale_Sensor_Read_All(g_sensor_data);

        for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
        {
            printf("[ %s:%d ]", sensor_labels[i], g_sensor_data[i]);
        }
        printf("\n");
        delay_ms(100);
    }
}

```

- `app_main`: ESP-IDF 项目的主入口函数。它首先调用 `Grayscale_Sensor_Init()` 初始化传感器，然后使用 `xTaskCreate()` 创建一个专门用于读取和打印传感器数据的 FreeRTOS 任务。
- `Grayscale_ReadData_Task`: 一个无限循环的任务。在每次循环中，它调用 `Grayscale_Sensor_Read_All()` 获取最新的传感器数据，然后通过 `printf` 将数据格式化并输出到串口。

```

// grayscale_sensor.c
void Grayscale_Sensor_Init(void)
{
    gpio_config_t io_conf_output = {
        .pin_bit_mask = (1ULL << 35) | (1ULL << 36) | (1ULL << 37),
        .mode = GPIO_MODE_OUTPUT,
        //...
    };
    gpio_config(&io_conf_output);

    gpio_config_t io_conf_input = {
        .pin_bit_mask = (1ULL << 38),
        .mode = GPIO_MODE_INPUT,
        //...
    };
    gpio_config(&io_conf_input);
}

void Grayscale_Sensor_Read_All(uint16_t* sensor_values)
{
    uint8_t i;
    for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
    {
        _select_channel(i);
        _delay_us(50);
        sensor_values[i] = Read_OUT_value();
    }
}

```

- `Grayscale_Sensor_Init`: 使用 ESP-IDF 提供的 `gpio_config` 结构体和函数来初始化 GPIO。将控制引脚(GPIO35-37)配置为输出，数据引脚(GPIO38)配置为输入。

- `Grayscale_Sensor_Read_All`: 遍历所有8个传感器通道, 通过 `_select_channel` 选择通道, 然后读取其值并存入数组。